SHRIRANJINI

PES1UG24CS447

1A

```
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4
b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bb
e9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$
```

1B

```
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ find .pes/objects
-type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$
```

2A

```
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$
```

2B

```
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ xxd .pes/objects/b
d/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442 | head -20
00000000: 7472 6565 2034 3900 3130 3036 3434 2068  tree 49.100644 h
00000010: 656c 6c6f 2e74 7874 0066 2246 63d2 3e6f  ello.txt.f"Fc.>o
00000020: 4d9d e9e2 c7e6 d876 4305 a92a 3830 a1a5  M......vC..*80..
00000030: 2d3d 5f4a a800 7b5c 39                   -=_J..{\9
```

3A

```
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ ./pes add file1.tx
t file2.txt
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  commit.h
  untracked:  tree.h
  untracked:  tree.c
  untracked:  test_tree.c
  untracked:  object.c
  untracked:  pes.c
  untracked:  commit.c
  untracked:  pes.h
```

```
  untracked:  pes.c
  untracked:  commit.c
  untracked:  pes.h
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c.save
  untracked:  index.c
  untracked:  make_tree
  untracked:  file.txt
  untracked:  test_sequence.sh
  untracked:  README.md
  untracked:  make_tree.c
  untracked:  test_objects.c

shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$
```

3B

```
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776430
788 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776430
802 6 file2.txt
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$
```

4A

```
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ ./pes log
commit c72d6bf9d8e283eb20b88a36ef79eb85c59d42aa60ae6bb246042ac0cdf8bace
Author: PES User <pes@localhost>
Date:   1776431964

    Add farewall

commit 8fd63683f004fca09c5fe3c7915b34dd0d2d66f20a63a710fe87bee63b8c5647
Author: PES User <pes@localhost>
Date:   1776431903

    Add

commit 798752994b18915f9e23dc6e3b28a4767db4dc39a6a59f80970264a45e61a976
Author: PES User <pes@localhost>
Date:   1776431813

    Initial committial commiQ

shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$
```

4B

```
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ find .pes -type f
| sort
.pes/HEAD
.pes/index
.pes/objects/35/11f815fca26d89ac853480ca55a7cea2dbd869c50397a9d6bc02f1d5cc952f
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/77/491ae416cbe5b2ebc275d97d6c3de76bb8eeb55b1ce87150c0d529d1903880
.pes/objects/79/8752994b18915f9e23dc6e3b28a4767db4dc39a6a59f80970264a45e61a976
.pes/objects/8f/d63683f004fca09c5fe3c7915b34dd0d2d66f20a63a710fe87bee63b8c5647
.pes/objects/b3/d731c8f5ea38a1118f9300e2dc257dee11461d53f4a1340e6afc4878ebec12
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/c7/2d6bf9d8e283eb20b88a36ef79eb85c59d42aa60ae6bb246042ac0cdf8bace
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$
```

4C

```
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ cat .pes/refs/head
s/main
c72d6bf9d8e283eb20b88a36ef79eb85c59d42aa60ae6bb246042ac0cdf8bace
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$
```

```
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ make test-integrat
ion
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
```

```
--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 3fdef3ba7653... Initial commit

Log after first commit:
commit 3fdef3ba7653bb47dd11f19d32dee5a4fac42310f7cee0783be8a2782d4e2a99
Author: PES User <pes@localhost>
Date:   1776432380

    Initial commit
```

```
--- Second Commit ---
Committed: 609ac1ddb9c8... Update file.txt

--- Third Commit ---
Committed: ebb23dcac433... Add farewell

--- Full History ---
commit ebb23dcac43302c997ec0579b0f4d2bda2b73ed10d7f1658af3c07314c44621e
Author: PES User <pes@localhost>
Date:   1776432380

    Add farewell

commit 609ac1ddb9c83425dbac6e09c97a504ac80b7e4b472c019d28d2733f4a240bde
Author: PES User <pes@localhost>
Date:   1776432380

    Update file.txt

commit 3fdef3ba7653bb47dd11f19d32dee5a4fac42310f7cee0783be8a2782d4e2a99
Author: PES User <pes@localhost>
Date:   1776432380
```

```
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
ebb23dcac43302c997ec0579b0f4d2bda2b73ed10d7f1658af3c07314c44621e

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/3f/def3ba7653bb47dd11f19d32dee5a4fac42310f7cee0783be8a2782d4e2a99
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/60/9ac1ddb9c83425dbac6e09c97a504ac80b7e4b472c019d28d2733f4a240bde
.pes/objects/84/6ee3cf569e829c1c3e6fe4e93654c069c668da8ad258e837dd70e4c896e0f1
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/eb/b23dcac43302c997ec0579b0f4d2bda2b73ed10d7f1658af3c07314c44621e

=== All integration tests completed ===
shri@shri2006:~/PES1UG24CS447-pes-vcs/PES1UG24CS447-pes-vcs$ s
```

Q5.1)A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

1. Read the commit hash from:
   .pes/refs/heads/<branch>
2. Update HEAD:
   .pes/HEAD → ref: refs/heads/<branch>
3. Read the commit object → get its tree hash
4. Update working directory:
   a. Remove current tracked files
   b. Restore files from the tree using blobs from .pes/objects
5. Update .pes/index to match the checked-out tree

Q 5.2) When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

1 Use stat() to get:

- **file size**
- **modification time (mtime)**

2 Compare with stored index values

3 If mismatch:

- **Recompute file hash**
- **Compare with stored hash in index**

Conflict condition:

If:

- **Working file ≠ index version**
  AND
- **Target branch version is different**

Abort checkout (dirty working directory)

QQ5.3) "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

What happens:

- **New commits are created**
- **But no branch points to them**

Recovery:

- **Create a branch:**

  pes branch <name> <commit_hash>

These commits may be lost after garbage collection if not referenced.

Q6.1)Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Algorithm:

1. **Start from all references:**
   - **.pes/refs/heads**
   - **.pes/HEAD**
2. **Traverse object graph:**
   - **Commit → parent + tree**
   - **Tree → blobs + subtrees**
3. **Mark all reachable objects**
4. **Delete objects not marked**

Data structure:

Use a Hash Set

- **Fast lookup (O(1))**
- **Avoid revisiting objects**

Estimate:

100,000 commits

50 branches

Objects visited:

100,000 commits

100,000 trees

Many blobs

Total: Hundreds of thousands to millions of objects

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Race condition:

1. **Commit starts:**
    - **Creates new objects (blob, tree, commit)**
    - **Not yet linked to branch**
2. **GC runs:**
    - **Does not see new objects as reachable**
    - **Deletes them**
3. **Commit completes:**
    - **References deleted objects → corruption**

How Git avoids this:

Atomic updates: write objects first, update references last

Locking: prevent GC during commit

Temporary protection: new objects treated as reachable

Grace period: delay deletion of recent objects